

Anatolie Ernu

IT & Security Solutions
<https://www.ernu.eu>

GHID TEHNIC

Instalare și Hardening WordPress

Debian 13 Trixie · Nginx · MariaDB 10.11 · PHP 8.3-FPM

Versiune: 1.0 | 2026 | Enterprise Edition

Categorie	Detalii
Sistem de operare	Debian 13 (Trixie) – minimal install
Web server	Nginx 1.24+ (stable)
Bază de date	MariaDB 10.11 LTS
Runtime PHP	PHP 8.3-FPM
Firewall	UFW + Fail2Ban + ModSecurity (OWASP CRS)
TLS/SSL	Certbot (Let's Encrypt) sau certificat propriu
Actualizare automată	unattended-upgrades
Monitorizare	Wazuh Agent + auditd

Anatolie Ernu
IT & Security Solutions
<https://www.ernu.eu>

1. PREGĂTIREA SISTEMULUI DEBIAN 13

1.1 Instalare minimală Debian 13

Se recomandă folosirea imaginii ISO netinstall pentru a obține un sistem minimal fără pachete inutile.

 Descărcați `debian-13.x.x-amd64-netinst.iso` de pe <https://www.debian.org/distrib/netinst>

Opțiuni recomandate la instalare:

- Partiționare: / separată de /var și /home
- Software selection: selectați DOAR 'SSH server' și 'standard system utilities'
- NU instalați Desktop Environment, print server sau alte servicii
- Creați un utilizator non-root pentru administrare

1.2 Configurare inițială post-instalare

1.2.1 Actualizare completă a sistemului

```
# Actualizați lista de pachete și sistemul
apt update && apt upgrade -y

# Instalați pachete esențiale
apt install -y curl wget gnupg2 ca-certificates lsb-release \
  apt-transport-https software-properties-common unzip \
  vim htop net-tools ufw fail2ban auditd git
```

1.2.2 Configurare hostname și timezone

```
# Setati hostname-ul serverului
hostnamectl set-hostname web01.ernu.sec

# Setati timezone Moldova
timedatectl set-timezone Europe/Chisinau

# Activați sincronizare NTP
timedatectl set-ntp true
timedatectl status
```

1.2.3 Hardening SSH

 Modificați portul SSH înainte de a activa firewall-ul pentru a evita blocarea!

```
# Editați configurația SSH
nano /etc/ssh/sshd_config
```

Parametri obligatorii în /etc/ssh/sshd_config:

```
Port 2222          # Schimbați portul implicit
Protocol 2         # Forțați SSH v2
PermitRootLogin no  # Interziceți login root
PasswordAuthentic no # Doar autentificare cu cheie
PubkeyAuthenticat yes
AuthorizedKeysFile .ssh/authorized_keys
MaxAuthTries 3     # Max încercări autentificare
MaxSessions 5
LoginGraceTime 30
ClientAliveInterval 300
ClientAliveCountMax 2
X11Forwarding no
AllowAgentForwarding no
AllowTcpForwarding no
UseDNS no
Banner /etc/ssh/banner.txt
AllowUsers adminuser # Înlocuiți cu userul vostru
```

```
# Creați banner de avertizare
echo 'Acces autorizat exclusiv. Toate sesiunile sunt monitorizate.' > /etc/ssh/banner.txt

# Restart SSH
systemctl restart sshd
```

✓ SSH este acum configurat să accepte doar chei publice pe port non-standard

1.2.4 Configurare UFW Firewall

```
# Resetați regulile UFW
ufw --force reset

# Politică implicită: refuză tot
ufw default deny incoming
ufw default allow outgoing

# Permiteți SSH pe noul port
ufw allow 2222/tcp comment 'SSH'

# Permiteți HTTP și HTTPS
ufw allow 80/tcp comment 'HTTP'
ufw allow 443/tcp comment 'HTTPS'

# Activați UFW
ufw --force enable
ufw status verbose
```

1.2.5 Configurare kernel – parametri sysctl hardening

```
# Creați fișier de configurare sysctl hardening
cat > /etc/sysctl.d/99-hardening.conf << 'EOF'
# Protecție IP Spoofing
net.ipv4.conf.all.rp_filter = 1
net.ipv4.conf.default.rp_filter = 1

# Dezactivare redirectionare pachete IP
net.ipv4.ip_forward = 0
net.ipv4.conf.all.send_redirects = 0
net.ipv4.conf.default.send_redirects = 0

# Ignorare ICMP redirect
net.ipv4.conf.all.accept_redirects = 0
net.ipv4.conf.default.accept_redirects = 0
net.ipv6.conf.all.accept_redirects = 0

# Protecție SYN Flood
net.ipv4.tcp_syncookies = 1
net.ipv4.tcp_max_syn_backlog = 2048
net.ipv4.tcp_synack_retries = 2
net.ipv4.tcp_syn_retries = 5

# Dezactivare ping (opțional)
net.ipv4.icmp_echo_ignore_all = 0
net.ipv4.icmp_echo_ignore_broadcasts = 1

# Protecție buffer overflow
kernel.randomize_va_space = 2
kernel.exec-shield = 1
fs.protected_hardlinks = 1
fs.protected_symlinks = 1
EOF

# Aplicați setările
```

```
systemctl --system
```

1.2.6 Actualizări automate de securitate

```
apt install -y unattended-upgrades apt-listchanges

# Configurați actualizările automate
cat > /etc/apt/apt.conf.d/50unattended-upgrades << 'EOF'
Unattended-Upgrade::Allowed-Origins {
    "${distro_id}:${distro_codename}-security";
    "${distro_id}ESMApps:${distro_codename}-apps-security";
};
Unattended-Upgrade::AutoFixInterruptedDpkg "true";
Unattended-Upgrade::Remove-Unused-Dependencies "true";
Unattended-Upgrade::Automatic-Reboot "false";
Unattended-Upgrade::Mail "admin@ernu.sec";
EOF

dpkg-reconfigure -pmedium unattended-upgrades
```

2. INSTALARE ȘI CONFIGURARE MARIADB 10.11

2.1 Instalare MariaDB

```
# Adăugați repository-ul oficial MariaDB
curl -Ls https://downloads.mariadb.com/MariaDB/mariadb_repo_setup \
| bash -s -- --mariadb-server-version=10.11

# Instalați MariaDB
apt update
apt install -y mariadb-server mariadb-client

# Activați și porniți serviciul
systemctl enable --now mariadb
systemctl status mariadb
```

2.2 Securizare MariaDB (mysql_secure_installation)

 Rulați scriptul de securizare interactivă imediat după instalare

```
mysql_secure_installation
```

Răspundeți la întrebări astfel:

```
Enter current password for root: [Enter] # la prima rulare e gol
Switch to unix_socket authentication: Y
Change the root password: Y # setați o parolă puternică
Remove anonymous users: Y
Disallow root login remotely: Y
Remove test database: Y
Reload privilege tables: Y
```

2.3 Creare bază de date și utilizator WordPress

 Folosiți parole de minimum 24 de caractere cu litere mari/mici, cifre și caractere speciale!

```
# Conectați-vă la MariaDB ca root
mysql -u root -p

-- Înlocuiți 'PAROLA_PUTERNICA_DB' cu o parolă reală
CREATE DATABASE wp_site1 CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'wp_user1'@'localhost' IDENTIFIED BY 'PAROLA_PUTERNICA_DB';
GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, DROP, INDEX, ALTER
    ON wp_site1.* TO 'wp_user1'@'localhost';
FLUSH PRIVILEGES;

-- Verificați utilizatorul creat
SELECT User, Host, plugin FROM mysql.user WHERE User='wp_user1';
SHOW GRANTS FOR 'wp_user1'@'localhost';
EXIT;
```

2.4 Hardening configurație MariaDB

```
# Editați configurația principală
nano /etc/mysql/mariadb.conf.d/50-server.cnf
```

Adăugați / modificați acești parametri în secțiunea [mysqld]:

```
[mysqld]
# Dezactivați accesul de la rețea (WordPress e pe același server)
bind-address      = 127.0.0.1
skip-networking   = 0

# Securitate
local-infile      = 0
symbolic-links    = 0
skip-show-database

# Performance și stabilitate
max_connections   = 100
max_allowed_packet = 64M
tmp_table_size    = 32M
max_heap_table_size = 32M

# Logging pentru audit
general_log       = 0
slow_query_log    = 1
slow_query_log_file = /var/log/mysql/slow-query.log
long_query_time   = 2

# Character set
character-set-server = utf8mb4
collation-server     = utf8mb4_unicode_ci

# Restart MariaDB
systemctl restart mariadb
systemctl status mariadb
```

 MariaDB este configurat să accepte conexiuni doar de pe localhost

3. INSTALARE ȘI CONFIGURARE PHP 8.3-FPM

3.1 Adăugare repository PHP și instalare

```
# Adăugați repository Sury (pachete PHP actualizate)
curl -sLo /usr/share/keyrings/deb.sury.org-php.gpg \
  https://packages.sury.org/php/apt.gpg

echo "deb [signed-by=/usr/share/keyrings/deb.sury.org-php.gpg] \
  https://packages.sury.org/php/ $(lsb_release -sc) main" \
  > /etc/apt/sources.list.d/php.list

apt update

# Instalați PHP 8.3 și extensiile necesare pentru WordPress
apt install -y php8.3-fpm php8.3-mysql php8.3-xml php8.3-gd \
  php8.3-mbstring php8.3-curl php8.3-zip php8.3-intl \
  php8.3-bcmath php8.3-soap php8.3-imagick php8.3-redis \
  php8.3-openssl

# Verificați versiunea
php8.3 --version
php8.3-fpm --version
```

3.2 Hardening php.ini

```
# Faceți backup la configurația originală
cp /etc/php/8.3/fpm/php.ini /etc/php/8.3/fpm/php.ini.bak

# Editați configurația PHP-FPM
nano /etc/php/8.3/fpm/php.ini
```

Parametri de securitate obligatorii în php.ini:

```
; — SECURITATE GENERALĂ —
expose_php = Off          ; Ascunde versiunea PHP din header
display_errors = Off      ; Nu afișa erori în browser
display_startup_errors = Off
log_errors = On           ; Logați erorile
error_log = /var/log/php8.3-fpm/php_errors.log
error_reporting = E_ALL & ~E_DEPRECATED & ~E_STRICT

; — FUNCȚII PERICULOASE —
disable_functions = exec,passthru,shell_exec,system,proc_open,\
  popen,curl_exec,curl_multi_exec,parse_ini_file,show_source,\
  phpinfo,pcntl_alarm,pcntl_fork,pcntl_waitpid,pcntl_wait,\
  pcntl_wifexited,pcntl_wifstopped,pcntl_wifsignaled,\
  pcntl_wifcontinued,pcntl_wexitstatus,pcntl_wtermsig,\
  pcntl_wstopsig,pcntl_signal,pcntl_signal_get_handler,\
  pcntl_signal_dispatch,pcntl_get_last_error,pcntl_strerror,\
  pcntl_sigprocmask,pcntl_sigwaitinfo,pcntl_sigtimedwait,\
  pcntl_exec,pcntl_getpriority,pcntl_setpriority,pcntl_async_signals

; — UPLOAD ȘI MEMORIE —
file_uploads = On
upload_max_filesize = 64M
post_max_size = 64M
max_execution_time = 300
max_input_time = 300
memory_limit = 256M
max_input_vars = 3000
```

```
; — SESIUNI —
session.cookie_httponly = On
session.cookie_secure = On
session.cookie_samesite = Strict
session.use_strict_mode = On
session.use_only_cookies = On
session.gc_maxlifetime = 1440

; — OPENBASEDIR – restricționați accesul la sistem de fișiere
; open_basedir = /var/www/site1/tmp/usr/share/php
; (decomentați și adaptați după structura voastră de directoare)

; — OPCACHE —
opcache.enable = 1
opcache.memory_consumption = 128
opcache.interned_strings_buffer = 8
opcache.max_accelerated_files = 4000
opcache.revalidate_freq = 60
opcache.fast_shutdown = 1
opcache.enable_cli = 0
```

3.3 Configurare PHP-FPM Pool

Fiecare site WordPress trebuie să ruleze sub propriul pool FPM cu utilizator dedicat:

```
# Creați utilizator de sistem dedicat pentru site
useradd -r -s /usr/sbin/nologin -d /var/www/site1 wp_site1

# Creați pool FPM dedicat pentru site1
cat > /etc/php/8.3/fpm/pool.d/site1.conf << 'EOF'
[site1]
user = wp_site1
group = wp_site1
listen = /run/php/php8.3-site1.sock
listen.owner = www-data
listen.group = www-data
listen.mode = 0660

; Tip de gestionare a proceselor
pm = dynamic
pm.max_children = 20
pm.start_servers = 5
pm.min_spare_servers = 3
pm.max_spare_servers = 10
pm.max_requests = 500

; Securitate
security.limit_extensions = .php
php_admin_value[disable_functions] = exec,passthru,shell_exec,system
php_admin_flag[allow_url_fopen] = off
php_admin_flag[allow_url_include] = off
php_admin_value[open_basedir] = /var/www/site1/tmp
php_admin_value[error_log] = /var/log/php8.3-fpm/site1-error.log
php_admin_flag[log_errors] = on

; Variabile mediu
env[HOSTNAME] = $HOSTNAME
env[TMP] = /tmp
env[TMPDIR] = /tmp
env[TEMP] = /tmp
EOF

# Restart PHP-FPM
systemctl restart php8.3-fpm
systemctl status php8.3-fpm
```

4. INSTALARE ȘI HARDENING NGINX

4.1 Instalare Nginx

```
# Adăugați repository oficial Nginx
curl -fsSL https://nginx.org/keys/nginx_signing.key \
| gpg --dearmor -o /usr/share/keyrings/nginx.gpg

echo "deb [signed-by=/usr/share/keyrings/nginx.gpg] \
https://nginx.org/packages/debian $(lsb_release -sc) nginx" \
> /etc/apt/sources.list.d/nginx.list

apt update
apt install -y nginx

systemctl enable --now nginx
nginx -v
```

4.2 Configurare globală Nginx hardening

Editați fișierul principal de configurare:

```
# Backup configurație originală
cp /etc/nginx/nginx.conf /etc/nginx/nginx.conf.bak

cat > /etc/nginx/nginx.conf << 'EOF'
user www-data;
worker_processes auto;
worker_rlimit_nofile 65535;
pid /run/nginx.pid;

events {
    worker_connections 1024;
    multi_accept on;
    use epoll;
}

http {
    # — Bază —
    include /etc/nginx/mime.types;
    default_type application/octet-stream;
    charset utf-8;

    # — Ascunde versiunea Nginx —
    server_tokens off;

    # — Performance —
    sendfile on;
    tcp_nopush on;
    tcp_nodelay on;
    keepalive_timeout 65;
    types_hash_max_size 2048;

    # — Limitare request —
    client_body_buffer_size 16K;
    client_header_buffer_size 1k;
    client_max_body_size 64m;
    large_client_header_buffers 4 16k;
    client_body_timeout 12;
    client_header_timeout 12;
    send_timeout 10;
```

```
# — Rate Limiting – protecție DDoS/brute-force —
limit_req_zone $binary_remote_addr zone=general:10m rate=30r/m;
limit_req_zone $binary_remote_addr zone=login:10m rate=5r/m;
limit_req_zone $binary_remote_addr zone=xmlrpc:10m rate=2r/m;
limit_conn_zone $binary_remote_addr zone=addr:10m;

# — Gzip —
gzip on;
gzip_vary on;
gzip_proxied any;
gzip_comp_level 6;
gzip_types text/plain text/css text/xml text/javascript
application/json application/javascript application/xml
application/rss+xml font/ttf image/svg+xml;

# — Logging —
log_format main '$remote_addr - $remote_user [$time_local]'
'"$request" $status $body_bytes_sent'
'"$http_referer" "$http_user_agent"'
'"$http_x_forwarded_for" rt=$request_time';
access_log /var/log/nginx/access.log main;
error_log /var/log/nginx/error.log warn;

# — Include virtual hosts —
include /etc/nginx/conf.d/*.conf;
include /etc/nginx/sites-enabled/*;
}
EOF
```

4.3 Configurare Virtual Host WordPress cu TLS

⚠️ Înlocuiți wp01.ernu.sec cu domeniul vostru real înainte de a aplica!

```
# Instalați Certbot pentru Let's Encrypt
apt install -y certbot python3-certbot-nginx

# Obțineți certificat SSL (rulați DUPĂ ce Nginx pornește pe port 80)
certbot --nginx -d wp01.ernu.sec -d wp01.ernu.sec \
--email admin@ernu.sec --agree-tos --no-eff-email
```

```
# Creați fișier Virtual Host pentru WordPress
cat > /etc/nginx/sites-available/site1.conf << 'NGINXEOF'

# Redirecționare HTTP -> HTTPS
server {
    listen 80;
    listen [::]:80;
    server_name wp01.ernu.sec wp01.ernu.sec;
    return 301 https://$host$request_uri;
}

# Server HTTPS principal
server {
    listen 443 ssl http2;
    listen [::]:443 ssl http2;
    server_name wp01.ernu.sec wp01.ernu.sec;
    root /var/www/site1;
    index index.php;

    # — TLS/SSL —
    ssl_certificate /etc/letsencrypt/live/wp01.ernu.sec/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/wp01.ernu.sec/privkey.pem;
    ssl_session_timeout 1d;
```

```

ssl_session_cache shared:MozSSL:10m;
ssl_session_tickets off;

# TLS modern (Mozilla Intermediate)
ssl_protocols TLSv1.2 TLSv1.3;
ssl_ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:
  ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:
  ECDHE-ECDSA-CHACHA20-POLY1305:ECDHE-RSA-CHACHA20-POLY1305:
  DHE-RSA-AES128-GCM-SHA256:DHE-RSA-AES256-GCM-SHA384;
ssl_prefer_server_ciphers off;

# HSTS – forțați HTTPS pentru 2 ani
add_header Strict-Transport-Security 'max-age=63072000; includeSubDomains; preload' always;

# — Security Headers —————
add_header X-Frame-Options      'SAMEORIGIN'      always;
add_header X-Content-Type-Options 'nosniff'        always;
add_header X-XSS-Protection    '1; mode=block'    always;
add_header Referrer-Policy      'strict-origin-when-cross-origin' always;
add_header Permissions-Policy   'camera=0,microphone=0,geolocation=0' always;
add_header Content-Security-Policy "default-src 'self'; script-src 'self' 'unsafe-inline' 'unsafe-eval'; style-src 'self' 'unsafe-inline'; img-src 'self' data: https:; font-src 'self' data:;" always;

# — Rate Limiting —————
limit_conn addr 20;

# — WordPress – Reguli principale —————
location / {
    limit_req zone=general burst=20 nodelay;
    try_files $uri $uri/ /index.php?$args;
}

# — Protecție wp-login.php —————
location = /wp-login.php {
    limit_req zone=login burst=3 nodelay;
    # Restricționați accesul la IP-uri cunoscute (decomentați)
    # allow 10.0.0.1; # IP-ul vostru de administrare
    # deny all;
    fastcgi_pass unix:/run/php/php8.3-site1.sock;
    fastcgi_index index.php;
    fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
    include fastcgi_params;
}

# — Blocați xmlrpc.php —————
location = /xmlrpc.php {
    limit_req zone=xmlrpc burst=1 nodelay;
    deny all;
    return 403;
}

# — Procesare PHP —————
location ~ \.php$ {
    # Preveniți execuția PHP în upload-uri
    try_files $uri =404;
    fastcgi_split_path_info ^(.+\.php)(/.+)$;
    fastcgi_pass unix:/run/php/php8.3-site1.sock;
    fastcgi_index index.php;
    fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
    include fastcgi_params;
    fastcgi_intercept_errors on;
    fastcgi_buffer_size 128k;
    fastcgi_buffers 256 16k;
    fastcgi_busy_buffers_size 256k;
    fastcgi_read_timeout 300;
}

# — Blocați accesul la fișiere sensibile —————
location ~ /\. { deny all; }
location ~ /wp-config\.php { deny all; }
location ~ /wp-config-sample\.php { deny all; }
location ~* \.(?:(log|bak|sql|tar|gz|zip|env|ini|conf)$ { deny all; }

```

```
location ~* /(?:readme|license|changelog|readme\.html)$ { deny all; }

# — Blocați execuția PHP în uploads —————
location ~* /wp-content/uploads/.*\.php$ {
    deny all;
    return 403;
}

# — Static files – cache agresiv —————
location ~* \.(?:css|js|jpg|jpeg|gif|png|ico|svg|woff|woff2|ttf|eot)$ {
    expires 30d;
    add_header Cache-Control 'public, no-transform';
    access_log off;
}

# — Logs per site —————
access_log /var/log/nginx/site1-access.log main;
error_log /var/log/nginx/site1-error.log warn;
}
NGINXEOF

# Activați site-ul
ln -s /etc/nginx/sites-available/site1.conf /etc/nginx/sites-enabled/

# Testați configurația
nginx -t

# Reload Nginx
systemctl reload nginx
```



Virtual Host configurat cu TLS 1.2/1.3, security headers și rate limiting

5. INSTALARE WORDPRESS ȘI HARDENING

5.1 Descărcare și instalare WordPress

```
# Creați directorul pentru site
mkdir -p /var/www/site1

# Descărcați ultima versiune WordPress
cd /tmp
wget https://wordpress.org/latest.tar.gz
tar -xzf latest.tar.gz

# Copiați fișierele
cp -r wordpress/* /var/www/site1/

# Setați proprietarul corect
chown -R wp_site1:wp_site1 /var/www/site1/

# Setați permisiunile corecte
find /var/www/site1 -type d -exec chmod 755 {} \;
find /var/www/site1 -type f -exec chmod 644 {} \;

# Permisuni speciale pentru directoare cu scriere
chmod 755 /var/www/site1/wp-content
chmod 755 /var/www/site1/wp-content/uploads
chmod 755 /var/www/site1/wp-content/themes
chmod 755 /var/www/site1/wp-content/plugins
```

5.2 Configurare wp-config.php hardened

```
# Copiați template-ul de configurare
cp /var/www/site1/wp-config-sample.php /var/www/site1/wp-config.php

# Generați chei de securitate unice (output-ul îl inserați în wp-config.php)
curl -s https://api.wordpress.org/secret-key/1.1/salt/
```

Editați /var/www/site1/wp-config.php cu parametrii de mai jos:

```
<?php
// — Baza de date —————
define('DB_NAME', 'wp_site1');
define('DB_USER', 'wp_user1');
define('DB_PASSWORD', 'PAROLA_PUTERNICA_DB');
define('DB_HOST', '127.0.0.1');
define('DB_CHARSET', 'utf8mb4');
define('DB_COLLATE', 'utf8mb4_unicode_ci');

// — Prefix tabel personalizat (NU folosiți wp_) —————
$table_prefix = 'x7r2_'; // generați aleator

// — Chei de autentificare (inserați output-ul de mai sus) —
define('AUTH_KEY', 'cheie-unica-1');
define('SECURE_AUTH_KEY', 'cheie-unica-2');
define('LOGGED_IN_KEY', 'cheie-unica-3');
define('NONCE_KEY', 'cheie-unica-4');
define('AUTH_SALT', 'cheie-unica-5');
define('SECURE_AUTH_SALT', 'cheie-unica-6');
define('LOGGED_IN_SALT', 'cheie-unica-7');
define('NONCE_SALT', 'cheie-unica-8');

// — Securitate —————
define('DISALLOW_FILE_EDIT', true); // Dezactivați editorul din admin
define('DISALLOW_FILE_MODS', false); // Permiteți update plugin/teme
define('FORCE_SSL_ADMIN', true); // Admin doar pe HTTPS
define('WP_DEBUG', false); // NU în producție!
define('WP_DEBUG_LOG', false);
define('WP_DEBUG_DISPLAY', false);
define('SCRIPT_DEBUG', false);

// — Limitare revizii post —————
define('WP_POST_REVISIONS', 5);

// — Auto-update —————
define('WP_AUTO_UPDATE_CORE', true);

// — Protecție directoare —————
define('WP_CONTENT_DIR', '/var/www/site1/wp-content');
define('WP_CONTENT_URL', 'https://wp01.ernu.sec/wp-content');

// — ABSPATH —————
if (!defined('ABSPATH')) define('ABSPATH', __DIR__ . '/');
require_once ABSPATH . 'wp-settings.php';

# Protejați wp-config.php cu permisiuni restrictive
chmod 640 /var/www/site1/wp-config.php
chown wp_site1:www-data /var/www/site1/wp-config.php
```

5.3 Protecție .htaccess echivalent prin Nginx

i Regulele de securitate Nginx din Capitolul 4 acoperă deja protecția echivalentă cu .htaccess WordPress. Nu este necesară configurație suplimentară.

5.4 Finalizare instalare WordPress

```
# Verificați că toate serviciile rulează
systemctl status nginx
systemctl status php8.3-fpm
systemctl status mariadb

# Accesați în browser URL-ul site-ului pentru a rula wizard-ul de instalare
# https://wp01.ernu.sec/wp-admin/install.php
```

! La instalarea WordPress: alegeți un username de administrator care NU este 'admin'! Folosiți o parolă de minimum 20 caractere generată aleator.

6. CONFIGURARE FAIL2BAN

6.1 Instalare și configurare de bază

```
# Instalați Fail2Ban
apt install -y fail2ban

# Copiați configurația locală (nu modificați jail.conf direct!)
cp /etc/fail2ban/jail.conf /etc/fail2ban/jail.local
```

6.2 Configurare jail-uri personalizate

```
cat > /etc/fail2ban/jail.d/wordpress.local << 'EOF'
[DEFAULT]
bantime = 3600          ; ban 1 oră
findtime = 600          ; fereastră 10 minute
maxretry = 5            ; 5 încercări eșuate
banaction = ufw

[sshd]
enabled = true
port = 2222
filter = sshd
logpath = /var/log/auth.log
maxretry = 3
bantime = 86400         ; ban 24 ore pentru SSH

[nginx-http-auth]
enabled = true
port = http,https
filter = nginx-http-auth
logpath = /var/log/nginx/error.log

[nginx-limit-req]
enabled = true
port = http,https
filter = nginx-limit-req
logpath = /var/log/nginx/error.log
```

```
maxretry = 10

[wordpress-auth]
enabled = true
port    = http,https
filter  = wordpress-auth
logpath = /var/log/nginx/site1-access.log
maxretry = 5
bantime = 7200
EOF
```

```
# Creați filtru custom pentru WordPress
cat > /etc/fail2ban/filter.d/wordpress-auth.conf << 'EOF'
[Definition]
failregex = ^<HOST> .* "POST /wp-login\.php
           ^<HOST> .* "POST /xmlrpc\.php
ignoreregex =
datepattern = {%LN-BEG}%d/%b/%Y:%H:%M:%S %z
EOF

# Restart și verificare Fail2Ban
systemctl enable --now fail2ban
systemctl status fail2ban
fail2ban-client status
```

✓ Fail2Ban protejează SSH, Nginx și wp-login.php împotriva atacurilor brute-force

7. PROTECȚIE AVANSATĂ – ModSecurity + OWASP CRS

7.1 Compilare ModSecurity cu Nginx

i ModSecurity v3 (libmodsecurity) funcționează cu Nginx prin modulul dinamic nginx-module-modsecurity

```
# Instalați dependențele pentru compilare
apt install -y libmodsecurity3 libmodsecurity-dev

# Instalați conectorul Nginx-ModSecurity
apt install -y libnginx-mod-http-modsecurity

# Activați modulul în Nginx
echo 'load_module /usr/lib/nginx/modules/ngx_http_modsecurity_module.so;' \
> /etc/nginx/modules-enabled/50-mod-http-modsecurity.conf
```

7.2 Descărcare OWASP Core Rule Set

```
# Descărcăți OWASP CRS
cd /tmp
wget https://github.com/coreruleset/coreruleset/archive/refs/tags/v3.3.5.tar.gz
tar -xzf v3.3.5.tar.gz
mv coreruleset-3.3.5 /etc/nginx/modsec-crs

# Configurați CRS
cp /etc/nginx/modsec-crs/crs-setup.conf.example \
/etc/nginx/modsec-crs/crs-setup.conf

# Copiați configurația ModSecurity
```

```
cp /etc/modsecurity/modsecurity.conf-recommended \
/etc/modsecurity/modsecurity.conf

# Activați ModSecurity (schimbați din DetectionOnly în On)
sed -i 's/SecRuleEngine DetectionOnly/SecRuleEngine On/' \
/etc/modsecurity/modsecurity.conf

# Creați fișier principal ModSecurity pentru Nginx
cat > /etc/nginx/modsec/main.conf << 'EOF'
Include /etc/modsecurity/modsecurity.conf
Include /etc/nginx/modsec-crs/crs-setup.conf
Include /etc/nginx/modsec-crs/rules/*.conf
EOF

mkdir -p /etc/nginx/modsec

# Adăugați în blocul server din Virtual Host:
# modsecurity on;
# modsecurity_rules_file /etc/nginx/modsec/main.conf;

# Testați și reîncărcați Nginx
nginx -t && systemctl reload nginx
```

8. PERMISIUNI FIȘIERE ȘI SISTEM

8.1 Script de verificare și setare permisiuni

```
#!/bin/bash
# Script: fix-wp-permissions.sh
# Rulați după fiecare instalare sau update major

WP_PATH="/var/www/site1"
WP_USER="wp_site1"
WEB_GROUP="www-data"

echo '=== Setare permisiuni WordPress ==='

# Proprietar
chown -R ${WP_USER}:${WEB_GROUP} ${WP_PATH}

# Directoare: 755 (rwxr-xr-x)
find ${WP_PATH} -type d -exec chmod 755 {} \;

# Fișiere: 644 (rw-r--r--)
find ${WP_PATH} -type f -exec chmod 644 {} \;

# wp-config.php: 640 (rw-r-----)
chmod 640 ${WP_PATH}/wp-config.php

# .htaccess: 444 (r--r--r--) – read-only
[ -f ${WP_PATH}/.htaccess ] && chmod 444 ${WP_PATH}/.htaccess

echo '=== Verificare permisiuni wp-config.php ==='
ls -la ${WP_PATH}/wp-config.php

echo '=== Done! ==='

# Salvați și rulați scriptul
```

```
chmod +x /usr/local/bin/fix-wp-permissions.sh
/usr/local/bin/fix-wp-permissions.sh
```

8.2 Protecție directoare cu permisiuni speciale

```
# Preveniți accesul direct la fișiere PHP în uploads
# (configurat deja în Nginx Virtual Host)

# Dezactivați listarea directorilor
# În Nginx: autoindex off; (implicit off)

# Verificați că nu există fișiere PHP în uploads
find /var/www/site1/wp-content/uploads -name '*.php' -exec rm {} \;

# Verificați permisiunile greșite
find /var/www/site1 -type f -perm /o+w -exec ls -la {} \;
```

9. BACKUP AUTOMAT

9.1 Script de backup complet

```
#!/bin/bash
# Script: wp-backup.sh – Backup WordPress + MariaDB

SITE_NAME="site1"
WP_PATH="/var/www/site1"
DB_NAME="wp_site1"
DB_USER="wp_user1"
DB_PASS="PAROLA_PUTERNICA_DB"
BACKUP_DIR="/var/backups/wordpress"
RETENTION_DAYS=30
DATE=$(date +%Y%m%d_%H%M%S)

mkdir -p ${BACKUP_DIR}

echo "[${date}] Incepere backup ${SITE_NAME}"

# 1. Backup baza de date
mysqldump -u${DB_USER} -p${DB_PASS} --single-transaction \
--quick --lock-tables=false ${DB_NAME} \
| gzip > ${BACKUP_DIR}/${SITE_NAME}_db_${DATE}.sql.gz

# 2. Backup fișiere WordPress
tar -czf ${BACKUP_DIR}/${SITE_NAME}_files_${DATE}.tar.gz \
--exclude='*.log' \
-C /var/www ${SITE_NAME}/

# 3. Ștergeți backup-uri mai vechi de RETENTION_DAYS zile
find ${BACKUP_DIR} -name "${SITE_NAME}_*.gz" \
-mtime +${RETENTION_DAYS} -delete

echo "[${date}] Backup finalizat: ${BACKUP_DIR}"
ls -lh ${BACKUP_DIR}/${SITE_NAME}_*_${DATE}*

# Salvați scriptul și setați permisiunile
chmod 700 /usr/local/bin/wp-backup.sh
```

```
# Adăugați în crontab (backup zilnic la 02:00)
crontab -e
# Adăugați linia:
0 2 * * * /usr/local/bin/wp-backup.sh >> /var/log/wp-backup.log 2>&1
```

10. MONITORIZARE ȘI AUDIT

10.1 Configurare auditd

```
# Instalați auditd
apt install -y auditd audispd-plugins

# Adăugați reguli de audit pentru WordPress
cat >> /etc/audit/rules.d/wordpress.rules << 'EOF'
# Monitorizați modificări wp-config.php
-w /var/www/site1/wp-config.php -p wa -k wp_config_change

# Monitorizați directorul uploads (execuție fișiere)
-w /var/www/site1/wp-content/uploads/ -p x -k wp_uploads_exec

# Monitorizați fișiere PHP noi în uploads
-w /var/www/site1/wp-content/uploads/ -p wa -k wp_uploads_write

# Monitorizați modificări la fișierele core WordPress
-w /var/www/site1/wp-admin/ -p wa -k wp_admin_change
-w /var/www/site1/wp-includes/ -p wa -k wp_includes_change
EOF

# Aplicați regulile
augenrules --load
systemctl restart auditd

# Verificați regulile active
auditctl -l
```

10.2 Verificare integritate fișiere WordPress (WP-CLI)

```
# Instalați WP-CLI
curl -O https://raw.githubusercontent.com/wp-cli/builds/gh-pages/phar/wp-cli.phar
chmod +x wp-cli.phar
mv wp-cli.phar /usr/local/bin/wp

# Verificați integritatea core WordPress
sudo -u wp_site1 wp --path=/var/www/site1 core verify-checksums

# Verificați plugin-uri
sudo -u wp_site1 wp --path=/var/www/site1 plugin verify-checksums --all

# Listați update-uri disponibile
sudo -u wp_site1 wp --path=/var/www/site1 core check-update
sudo -u wp_site1 wp --path=/var/www/site1 plugin list --update=available

# Script automatizat de verificare zilnică
cat > /usr/local/bin/wp-integrity-check.sh << 'EOF'
#!/bin/bash
echo "[$(date)] Verificare integritate WordPress"
sudo -u wp_site1 wp --path=/var/www/site1 core verify-checksums 2>&1
sudo -u wp_site1 wp --path=/var/www/site1 plugin verify-checksums --all 2>&1
EOF
chmod 700 /usr/local/bin/wp-integrity-check.sh
```

```
# Adăugați în crontab (verificare zilnică la 06:00)
# 0 6 * * * /usr/local/bin/wp-integrity-check.sh >> /var/log/wp-integrity.log 2>&1
```

11. CHECKLIST FINAL DE VERIFICARE

11.1 Verificare rapidă sistem

```
#!/bin/bash
# Script verificare rapidă post-instalare

echo '=====
echo ' VERIFICARE SECURITATE SERVER '
echo '=====

# Servicii active
echo "
echo '--- Servicii ---'
systemctl is-active nginx php8.3-fpm mariadb fail2ban ufw auditd

# UFW status
echo "
echo '--- Firewall UFW ---'
ufw status

# Fail2Ban jailuri
echo "
echo '--- Fail2Ban ---'
fail2ban-client status

# TLS check
echo "
echo '--- Certificat SSL ---'
openssl s_client -connect wp01.ernu.sec:443 </dev/null 2>&1 \
| grep -E 'subject|issuer|Verify'

# Permisuni wp-config.php
echo "
echo '--- Permisuni wp-config.php ---'
stat -c '%a %U:%G %n' /var/www/site1/wp-config.php

# Fișiere PHP în uploads
echo "
echo '--- Fișiere PHP în uploads ---'
find /var/www/site1/wp-content/uploads -name '*.php' 2>/dev/null \
  && echo 'ATENȚIE: fișiere PHP găsite!' \
  || echo 'OK: niciun fișier PHP'

echo "
echo '=====
echo ' VERIFICARE COMPLETĂ '
echo '=====
```

11.2 Tabel rezumat securitate

Componentă	Măsură de securitate	Status
Debian OS	Actualizări automate de securitate active	✓

SSH	Port non-standard, fără root login, doar chei SSH	✓
UFW Firewall	Deny all implicit, allow 2222/80/443 explicit	✓
Sysctl	Protecție SYN flood, IP spoofing, ASLR activ	✓
MariaDB	Bind localhost, fără root remote, parole puternice	✓
PHP 8.3-FPM	disable_functions, open_basedir, opcache, sesiuni securizate	✓
Nginx	server_tokens off, rate limiting, security headers, TLS 1.2/1.3	✓
TLS/SSL	Certbot Let's Encrypt, HSTS, ciphers moderne	✓
WordPress	DISALLOW_FILE_EDIT, prefix tabel random, FORCE_SSL_ADMIN	✓
Fail2Ban	Protecție SSH, Nginx, wp-login.php	✓
ModSecurity	OWASP CRS v3.3 activ – protecție WAF	✓
Permisuni	wp-config.php 640, uploads fără PHP exec	✓
Backup	Backup zilnic DB + fișiere, retenție 30 zile	✓
Audit	auditd monitorizează wp-config.php, uploads, core files	✓
WP-CLI	Verificare integritate core + plugin-uri zilnic	✓

11.3 Plugin-uri WordPress recomandate pentru securitate

Plugin	Funcție	Observații
Wordfence Security	Firewall aplicație, malware scan, 2FA	Versiunea gratuită este suficientă
WP 2FA	Autentificare cu doi factori pentru admin	Obligatoriu pentru contul admin
UpdraftPlus	Backup suplimentar în cloud (S3, GDrive)	Backup off-site
WPS Hide Login	Schimbă URL-ul paginii de login	Împiedică atacuri automatizate
Disable XML-RPC	Dezactivează xmlrpc.php complet	Alternativă la blocarea Nginx
Query Monitor	Debug performanță (doar dev/staging)	Dezactivați în producție

12. AL DOILEA SITE WORDPRESS (site2)

Repetăți pașii de mai jos pentru cel de-al doilea site, înlocuind site1 cu site2:

```
# 1. Baza de date
mysql -u root -p -e "
CREATE DATABASE wp_site2 CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'wp_user2'@'localhost' IDENTIFIED BY 'ALTA_PAROLA_PUTERNICA';
GRANT SELECT,INSERT,UPDATE,DELETE,CREATE,DROP,INDEX,ALTER
ON wp_site2.* TO 'wp_user2'@'localhost';
FLUSH PRIVILEGES;
"

# 2. Utilizator sistem dedicat
useradd -r -s /usr/sbin/nologin -d /var/www/site2 wp_site2

# 3. Pool PHP-FPM dedicat
cp /etc/php/8.3/fpm/pool.d/site1.conf \
/etc/php/8.3/fpm/pool.d/site2.conf
sed -i 's/site1/site2/g; s/wp_site1/wp_site2/g' \
/etc/php/8.3/fpm/pool.d/site2.conf
systemctl restart php8.3-fpm

# 4. Virtual Host Nginx
```

```
cp /etc/nginx/sites-available/site1.conf \
  /etc/nginx/sites-available/site2.conf
sed -i 's/site1/site2/g' /etc/nginx/sites-available/site2.conf
ln -s /etc/nginx/sites-available/site2.conf \
  /etc/nginx/sites-enabled/
nginx -t && systemctl reload nginx
```

```
# 5. Certificat SSL
certbot --nginx -d wp02.ernu.sec \
  --email admin@ernu.sec --agree-tos --no-eff-email
```

```
# 6. Instalare WordPress
mkdir -p /var/www/site2
cp -r /tmp/wordpress/* /var/www/site2/
chown -R wp_site2:www-data /var/www/site2/
find /var/www/site2 -type d -exec chmod 755 {} \;
find /var/www/site2 -type f -exec chmod 644 {} \;
```

✓ Fiecare site rulează izolat: utilizator propriu, pool FPM propriu, bază de date proprie, Virtual Host propriu.

13. CLOUDFLARE + NGINX PROXY MANAGER – IP REAL ÎN LOGURI

13.1 Diagrama fluxului de trafic

Traficul parcurge trei niveluri înainte de a ajunge la WordPress:

```
[Vizitator]
| IP real: 85.x.x.x
v
[Cloudflare CDN / WAF]
| Adauga header: CF-Connecting-IP: 85.x.x.x
| Header: X-Forwarded-For: 85.x.x.x
| IP sursa spre NPM: 104.x.x.x (IP Cloudflare)
v
[Nginx Proxy Manager]
| Transmite: X-Real-IP, X-Forwarded-For
| IP sursa spre Nginx WP: 192.168.x.x (IP intern NPM)
v
[Nginx WordPress - serverul vostru]
| Fara configuratie: vede IP-ul NPM, NU IP-ul real!
| Cu configuratie corecta: vede 85.x.x.x (IP real)
v
[MariaDB + PHP-FPM]
```

⚠ Fara configuratia de mai jos, logurile Nginx si Fail2Ban vad doar IP-ul NPM (192.168.x.x) sau IP-urile Cloudflare (104.x.x.x) – niciodata IP-ul real al atacatorului!

13.2 Pasul 1 – Configurare Nginx Proxy Manager

13.2.1 Configuratie Advanced per Proxy Host in NPM

In interfata web NPM: Edit Proxy Host → tab Advanced → Custom Nginx Configuration:

```
# NPM Advanced Config (per Proxy Host)
```

```
# Transmiteti IP-ul real Cloudflare spre backend Nginx WP
proxy_set_header X-Real-IP      $http_cf_connecting_ip;
proxy_set_header X-Forwarded-For $http_cf_connecting_ip;
proxy_set_header X-Forwarded-Proto $scheme;
proxy_set_header Host           $host;
```

13.2.2 Configuratie globala NPM (optionala – Docker)

Daca aveti acces la serverul NPM, editati fisierul global /data/nginx/custom/http.conf:

```
# /data/nginx/custom/http.conf (pe serverul NPM)

# IP-uri Cloudflare declarate ca surse de incredere
set_real_ip_from 103.21.244.0/22;
set_real_ip_from 103.22.200.0/22;
set_real_ip_from 103.31.4.0/22;
set_real_ip_from 104.16.0.0/13;
set_real_ip_from 104.24.0.0/14;
set_real_ip_from 108.162.192.0/18;
set_real_ip_from 131.0.72.0/22;
set_real_ip_from 141.101.64.0/18;
set_real_ip_from 162.158.0.0/15;
set_real_ip_from 172.64.0.0/13;
set_real_ip_from 173.245.48.0/20;
set_real_ip_from 188.114.96.0/20;
set_real_ip_from 190.93.240.0/20;
set_real_ip_from 197.234.240.0/22;
set_real_ip_from 198.41.128.0/17;
set_real_ip_from 2400:cb00::/32;
set_real_ip_from 2606:4700::/32;
set_real_ip_from 2803:f800::/32;
real_ip_header CF-Connecting-IP;
real_ip_recursive on;
```

13.3 Pasul 2 – Snippet real-ip pe serverul Nginx WordPress

13.3.1 Creare fisier snippet reutilizabil

```
# Creati directorul snippets daca nu exista
mkdir -p /etc/nginx/snippets

# Creati snippet-ul cu IP-urile Cloudflare + NPM
cat > /etc/nginx/snippets/real-ip-cloudflare.conf << 'SNIPEOF'
# Generat automat – actualizati lunar cu scriptul din sectiunea 13.6
# IP-uri Cloudflare IPv4
set_real_ip_from 103.21.244.0/22;
set_real_ip_from 103.22.200.0/22;
set_real_ip_from 103.31.4.0/22;
set_real_ip_from 104.16.0.0/13;
set_real_ip_from 104.24.0.0/14;
set_real_ip_from 108.162.192.0/18;
set_real_ip_from 131.0.72.0/22;
set_real_ip_from 141.101.64.0/18;
set_real_ip_from 162.158.0.0/15;
set_real_ip_from 172.64.0.0/13;
set_real_ip_from 173.245.48.0/20;
set_real_ip_from 188.114.96.0/20;
set_real_ip_from 190.93.240.0/20;
set_real_ip_from 197.234.240.0/22;
set_real_ip_from 198.41.128.0/17;
# IP-uri Cloudflare IPv6
set_real_ip_from 2400:cb00::/32;
set_real_ip_from 2606:4700::/32;
set_real_ip_from 2803:f800::/32;
```

```
set_real_ip_from 2405:b500::/32;
set_real_ip_from 2405:8100::/32;
set_real_ip_from 2a06:98c0::/29;
set_real_ip_from 2c0f:f248::/32;
# IP intern Nginx Proxy Manager (inlocuiti cu IP-ul real)
set_real_ip_from 192.168.1.100;
set_real_ip_from 172.16.0.0/12;
set_real_ip_from 127.0.0.1;
# Headerul din care citim IP-ul real transmis de NPM
real_ip_header X-Real-IP;
real_ip_recursive on;
SNIPEOF

chmod 644 /etc/nginx/snippets/real-ip-cloudflare.conf
```

13.3.2 Actualizare Virtual Host site1.conf

Adaugati in blocul server HTTPS urmatoarele directive (dupa linia listen):

```
server {
    listen 443 ssl http2;
    server_name wp01.ernu.sec;
    root /var/www/site1;

    # — INCLUDERE IP REAL CLOUDFLARE VIA NPM —————
    include /etc/nginx/snippets/real-ip-cloudflare.conf;

    # — FORMAT LOG EXTINS CU DATE CLOUDFLARE —————
    # $remote_addr = IP real al vizitatorului (dupa real_ip_header)
    log_format cf_real '$remote_addr - $remote_user [$time_local]'
        '$request' '$status $body_bytes_sent'
        '"$http_referer" "$http_user_agent"'
        'cf=$http_cf_connecting_ip'
        'country=$http_cf_ipcountry'
        'ray=$http_cf_ray'
        'rt=$request_time';

    access_log /var/log/nginx/site1-access.log cf_real;
    error_log /var/log/nginx/site1-error.log warn;

    # ... restul configuratiei din Capitolul 4 ramane identic
}

# Testati configuratia
nginx -t

# Aplicati
systemctl reload nginx

# Verificati ca $remote_addr este IP-ul real al vizitatorului
tail -f /var/log/nginx/site1-access.log
# Corect: 85.204.x.x - - [07/Jun/2026:10:23:11 +0300] ...
# Gresit: 192.168.1.x - - [07/Jun/2026:10:23:11 +0300] ...
```

✓ \$remote_addr in loguri = IP-ul real al vizitatorului, nu IP-ul NPM sau Cloudflare

13.4 Pasul 3 – Restrictie acces direct (bypass Cloudflare)

Serverul Nginx WordPress trebuie sa accepte conexiuni NUMAI de la Nginx Proxy Manager. Altfel cineva poate ocoli Cloudflare WAF accesand direct IP-ul serverului.

```
# In blocul server HTTPS – adaugati restrictia de acces:
server {
    # ...
```

```
# — PERMITETI ACCESUL DOAR DIN NPM —————
# Inlocuiti cu IP-ul real al serverului NPM
allow 192.168.1.100; # IP NPM
allow 127.0.0.1;    # localhost
allow 172.16.0.0/12; # Docker bridge (daca NPM e Docker)
deny all;          # blocheaza orice altceva
# ...
}
```

Varianta alternativa prin UFW (recomandata daca NPM si WP sunt pe servere separate):

```
# Pe serverul WordPress – permiteti porturile 80/443 NUMAI din NPM

# Stergeti regulile publice existente
ufw delete allow 80/tcp
ufw delete allow 443/tcp

# Permiteti doar din IP-ul NPM
ufw allow from 192.168.1.100 to any port 80 comment 'NPM->WP HTTP'
ufw allow from 192.168.1.100 to any port 443 comment 'NPM->WP HTTPS'

ufw reload && ufw status verbose
```

⚠ Inlocuiti 192.168.1.100 cu IP-ul real al serverului vostru Nginx Proxy Manager!

13.5 Pasul 4 – Verificare Fail2Ban cu IP real

Cu configuratia de mai sus, Fail2Ban citeste automat IP-ul real din loguri deoarece \$remote_addr = IP real. Verificati ca jail-ul citeste logul corect:

```
# Verificati ca logpath din jail.d/wordpress.local este corect
grep logpath /etc/fail2ban/jail.d/wordpress.local
# Trebuie sa arate: logpath = /var/log/nginx/site1-access.log

# Testati parsarea IP-urilor din log
fail2ban-regex /var/log/nginx/site1-access.log \
/etc/fail2ban/filter.d/wordpress-auth.conf
# Trebuie sa gaseasca match-uri cu IP-uri publice reale

# Verificati IP-urile ban-uite curent
fail2ban-client status wordpress-auth

# Test manual ban/unban
fail2ban-client set wordpress-auth banip 1.2.3.4
fail2ban-client set wordpress-auth unbanip 1.2.3.4
```

✓ Fail2Ban baneaza IP-urile reale ale atacatorilor, nu IP-urile Cloudflare (104.x.x.x) sau NPM (192.168.x.x)

13.6 Pasul 5 – Actualizare automata IP-uri Cloudflare

Cloudflare adauga periodic noi range-uri de IP. Scriptul urmator le actualizeaza lunar:

```
cat > /usr/local/bin/update-cloudflare-ips.sh << 'CFEOF'
#!/bin/bash
SNIPPET='/etc/nginx/snippets/real-ip-cloudflare.conf'
NPM_IP='192.168.1.100'

echo "# Generat automat: $(date)" > $SNIPPET
```

```
echo '# IP-uri Cloudflare IPv4' >> $SNIPPET
curl -sf https://www.cloudflare.com/ips-v4 | while read ip; do
    echo "set_real_ip_from ${ip};" >> $SNIPPET
done

echo '# IP-uri Cloudflare IPv6' >> $SNIPPET
curl -sf https://www.cloudflare.com/ips-v6 | while read ip; do
    echo "set_real_ip_from ${ip};" >> $SNIPPET
done

echo '# Nginx Proxy Manager' >> $SNIPPET
echo "set_real_ip_from ${NPM_IP};" >> $SNIPPET
echo 'set_real_ip_from 172.16.0.0/12;' >> $SNIPPET
echo 'set_real_ip_from 127.0.0.1;' >> $SNIPPET
echo 'real_ip_header X-Real-IP;' >> $SNIPPET
echo 'real_ip_recursive on;' >> $SNIPPET

nginx -t && systemctl reload nginx
echo "[$(date)] IP-uri Cloudflare actualizate OK"
CEOF

chmod 700 /usr/local/bin/update-cloudflare-ips.sh
/usr/local/bin/update-cloudflare-ips.sh

# Adaugati in crontab - actualizare lunara la 03:00
(crontab -l 2>/dev/null; echo '0 3 1 * * /usr/local/bin/update-cloudflare-ips.sh >> /var/log/cf-update.log 2>&1') | crontab -
```

13.7 Tabel rezumat configuratii per nivel

Nivel	Componenta	Configuratie cheie	Efect
1 – Edge	Cloudflare	Proxy ON (orange cloud)	Adauga CF-Connecting-IP cu IP real
2 – Proxy	Nginx Proxy Manager	proxy_set_header X-Real-IP \$http_cf_connecting_ip	Transmite IP real spre Nginx WP
3 – Backend	Nginx WordPress	include real-ip-cloudflare.conf	\$remote_addr = IP real in loguri
3 – Backend	Fail2Ban	logpath = /var/log/nginx/site1-access.log	Baneaza IP-uri reale, nu proxy
3 – Backend	UFW	allow from IP_NPM port 80/443; deny all	Blocheaza bypass Cloudflare
Auto	Cron lunar	update-cloudflare-ips.sh	IP-uri Cloudflare mereu actualizate

ANEXA A. Repository GitHub si livrare proiect

Branding oficial: Anatolie Ernu - IT & Security Solutions - <https://www.ernu.eu>
Repository: wordpress-enterprise-security-platform

A.1 Structura repository

```
wordpress-enterprise-security-platform/
├── docs/
├── examples/
├── scripts/
├── templates/
├── .env.example
├── README.md
├── SECURITY.md
├── CHANGELOG.md
└── LICENSE
```

A.2 Comenzi de import GitHub

```
cd /opt/wordpress-enterprise-security-platform
git init
```

```
git add .
git commit -m "Initial release v1.0"
git branch -M main
git remote add origin https://github.com/anatolie-ernu/wordpress-enterprise-security-platform.git
git push -u origin main
git tag -a v1.0 -m "wordpress-enterprise-security-platform v1.0"
git push origin v1.0
```

A.3 Verificari obligatorii dupa instalare

Componenta	Comanda	Rezultat asteptat
Sistem	cat /etc/os-release	Debian 13
Firewall	ufw status verbose	Default deny incoming
Servicii	systemctl --failed	0 failed units
Nginx	nginx -t	syntax is ok
PHP-FPM	systemctl status php8.3-fpm	active/running
MariaDB	mysqladmin ping	mysqlld is alive

A.4 Politica de parole si secrete

IMPORTANT: Fisierul .env real nu se urca niciodata in GitHub. Se publica doar .env.example cu valori placeholder.

```
openssl rand -base64 32
openssl rand -hex 32
```

A.5 Domenii si hostnames standard

```
Branding public: https://www.ernu.eu
Domeniu tehnic documentatie: ernu.sec
Email administrativ exemplu: admin@ernu.sec
```

ANEXA B. Extindere Fail2Ban + Nginx WordPress Protection

Aceasta anexa integreaza componentele utile din arhiva suplimentara wordpress-nginx-fail2ban-hardening si le adapteaza la standardul proiectului wordpress-enterprise-security-platform.

B.1 Componente adaugate

```
templates/fail2ban/filter.d/wordpress-login.conf
templates/fail2ban/filter.d/wordpress-scan.conf
templates/fail2ban/filter.d/nginx-404.conf
templates/fail2ban/jail.d/wordpress-enterprise-extra.local
templates/nginx/wordpress-security-include.conf
scripts/install-wordpress-nginx-fail2ban.sh
checklists/security-checklist.md
```

B.2 Instalare rapida

```
cd /opt/wordpress-enterprise-security-platform
sudo bash scripts/install-wordpress-nginx-fail2ban.sh /var/log/nginx/access.log
fail2ban-client status wordpress-login
fail2ban-client status wordpress-scan
fail2ban-client status nginx-404
```

B.3 Filtre Fail2Ban

```
[wordpress-login]
failregex = ^<HOST> - .* "(GET|POST).*wp-login\.php.*" (200|302|401|403).*
```

```
[wordpress-scan]
failregex = ^<HOST> - .* "(GET|POST|HEAD).*wp-config\.php|\.env|\.git|xmlrpc\.php|phpmyadmin|adminer|wlwmanifest\.xml|wp-content/
debug\.log).*" *
```

```
[nginx-404]
failregex = ^<HOST> - .* "(GET|POST|HEAD) .*" 404 .*
```

B.4 Nginx security include

```
location = /xmlrpc.php {
    deny all;
    access_log off;
    log_not_found off;
    return 403;
}

location ~* /(\\.env|\\.git|wp-config\\.php|readme\\.html|license\\.txt|composer\\.json|lock|package(-lock)?\\.json) {
    deny all;
    access_log off;
    log_not_found off;
    return 403;
}

location ~* /wp-content/uploads/.*\\.php$ {
    deny all;
    access_log off;
    log_not_found off;
    return 403;
}
```

B.5 Checklist operational

- ☐ IP real in logurile Nginx este validat inainte de banare.
- ☐ wp-login.php este monitorizat de Fail2Ban.
- ☐ Scanarile catre .env, .git, wp-config.php si phpmyadmin sunt detectate.
- ☐ xmlrpc.php este blocat daca nu este folosit.
- ☐ PHP execution in uploads este blocat.
- ☐ fail2ban-client status afiseaza jail-urile active.

ANEXA C. Planificare arhitectura, nume, adrese IP si porturi

Aceasta anexa stabileste explicit modul de functionare atunci cand Nginx Proxy Manager, Nginx backend, WordPress, PHP-FPM si MariaDB ruleaza pe acelasi server Linux. Pe acelasi IP nu pot exista doua servicii care asculta simultan public pe 0.0.0.0:80 sau 0.0.0.0:443. Din acest motiv, Nginx Proxy Manager ramane singurul serviciu public pe 80/443, iar Nginx backend pentru site-urile WordPress ruleaza doar local pe porturi interne.

Regula de baza: Nginx Proxy Manager asculta public pe 80/443; site-urile WordPress asculta local pe 127.0.0.1:8081, 127.0.0.1:8082 etc.; certificatele SSL se genereaza in Nginx Proxy Manager prin Cloudflare DNS Challenge.

C.1 Tabel planificare server, servicii si adrese

Componenta	Hostname / Nume	Adresa / IP	Porturi	Rol
Server Linux gazda	srv-wp01.ernu.sec	10.10.10.20 / IP public NAT	22/2222 SSH; 80/443 prin NPM	Debian 13 care gazduieste toate serviciile
Nginx Proxy Manager	proxy01.ernu.sec / npm-app	10.10.10.20	80/TCP public; 443/TCP public; 10.10.10.20:81 LAN admin	Reverse proxy public, TLS, Cloudflare DNS Challenge
MariaDB comuna	db01 local	127.0.0.1 si host-gateway Docker	3306 local / Docker subnet	Aceeasi instanta MariaDB pentru WordPress si NPM, dar baze separate
Baza WordPress site 1	wp01_db	MariaDB db01	intern DB	Baza de date pentru wp01.ernu.sec
Baza WordPress site 2	wp02_db	MariaDB db01	intern DB	Baza de date pentru wp02.ernu.sec
Baza NPM	npm	MariaDB db01	intern DB	Baza separata pentru Nginx Proxy Manager
Nginx backend site 1	wp01.ernu.sec	127.0.0.1	8081/TCP local	Backend WordPress site 1, fara TLS local
Nginx backend site 2	wp02.ernu.sec	127.0.0.1	8082/TCP local	Backend WordPress site 2, fara TLS local
PHP-FPM site 1	php8.3-wp01.sock	/run/php/php8.3-wp01.sock	Unix socket	Executie PHP pentru site 1
PHP-FPM site 2	php8.3-wp02.sock	/run/php/php8.3-wp02.sock	Unix socket	Executie PHP pentru site 2
Cloudflare DNS	zona publica	Cloudflare	API DNS Token	DNS Challenge pentru certificate in NPM

C.2 Tabel Proxy Hosts in Nginx Proxy Manager

Domeniu	Scheme	Forward Hostname/IP	Forward Port	SSL
wp01.ernu.sec	http	127.0.0.1	8081	Let's Encrypt via Cloudflare DNS Challenge
wp02.ernu.sec	http	127.0.0.1	8082	Let's Encrypt via

				Cloudflare DNS Challenge
Admin NPM	http	10.10.10.20	81	Doar LAN/VPN/allow-list, nu Internet public

C.3 Flux logic trafic

```

Internet / Cloudflare
|
v
Nginx Proxy Manager pe serverul Linux
- 10.10.10.20:80
- 10.10.10.20:443
|
+--> wp01.ernu.sec -> http://127.0.0.1:8081 -> Nginx backend -> PHP-FPM wp01 -> MariaDB wp01_db
|
+--> wp02.ernu.sec -> http://127.0.0.1:8082 -> Nginx backend -> PHP-FPM wp02 -> MariaDB wp02_db

```

ANEXA D. MariaDB comuna pentru WordPress si Nginx Proxy Manager

Pentru simplitate operationala, pe acelasi server Linux se poate folosi aceeaasi instanta MariaDB pentru site-urile WordPress si pentru Nginx Proxy Manager. Totusi, nu se foloseste aceeaasi baza de date/schema. Se creeaza baze separate si utilizatori separati, astfel incat compromiterea unei aplicatii sa nu ofere acces direct la datele celeilalte.

D.1 Baze si utilizatori recomandati

Aplicatie	Baza de date	Utilizator DB	Host permis	Observatii
WordPress site 1	wp01_db	wp01_user	localhost	Folosit de /var/www/wp01
WordPress site 2	wp02_db	wp02_user	localhost	Folosit de /var/www/wp02
Nginx Proxy Manager	npm	npm_user	172.30.10.%	Acces din reteaua Docker npm_net

D.2 Configurare MariaDB pentru acces local si Docker subnet

Daca Nginx Proxy Manager ruleaza in container Docker si MariaDB ruleaza pe hostul Linux, containerul nu poate folosi 127.0.0.1 pentru a ajunge la MariaDB de pe host. De aceea se foloseste host.docker.internal mapat la host-gateway si o retea Docker cu subnet fix.

```

# Fisier recomandat:
nano /etc/mysql/mariadb.conf.d/50-server.cnf

[mysqld]
bind-address = 0.0.0.0
skip-networking = 0

# Restart MariaDB dupa modificare
systemctl restart mariadb
systemctl status mariadb

```

Deschiderea bind-address pe 0.0.0.0 trebuie compensata prin firewall si prin utilizatori MariaDB limitati pe host/subnet. Nu se creeaza utilizatori DB cu acces global fara parola puternica.

D.3 Creare baze de date pentru WordPress si NPM

```

mysql -u root -p

CREATE DATABASE wp01_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'wp01_user'@'localhost' IDENTIFIED BY 'PAROLA_COMPLEXA_WP01';
GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, DROP, INDEX, ALTER
ON wp01_db.* TO 'wp01_user'@'localhost';

CREATE DATABASE wp02_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'wp02_user'@'localhost' IDENTIFIED BY 'PAROLA_COMPLEXA_WP02';
GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, DROP, INDEX, ALTER
ON wp02_db.* TO 'wp02_user'@'localhost';

CREATE DATABASE npm CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'npm_user'@'172.30.10.%' IDENTIFIED BY 'PAROLA_COMPLEXA_NPM';

```

```
GRANT ALL PRIVILEGES ON npm.* TO 'npm_user'@'172.30.10.%';

FLUSH PRIVILEGES;
SHOW GRANTS FOR 'npm_user'@'172.30.10.%';
EXIT;
```

D.4 Firewall pentru MariaDB

```
# Permite acces MariaDB doar din subnetul Docker folosit de NPM
ufw allow from 172.30.10.0/24 to any port 3306 proto tcp comment 'MariaDB for NPM docker network'

# Verificare
ufw status verbose
ss -lntp | grep 3306
```

ANEXA E. docker-compose.yml pentru Nginx Proxy Manager cu MariaDB comuna

Acest docker-compose porneste numai containerul Nginx Proxy Manager. Baza de date nu mai este un container separat npm-db, ci este aceeași instanță MariaDB folosită și de site-urile WordPress, cu baza separată npm și utilizator separat npm_user.

E.1 Structura directoare

```
mkdir -p /opt/npm
cd /opt/npm
mkdir -p data letsencrypt logs
chmod 750 data letsencrypt logs
```

E.2 Fisier .env

```
cat > /opt/npm/.env << 'EOF'
TZ=Europe/Chisinau
NPM_DB_HOST=host.docker.internal
NPM_DB_PORT=3306
NPM_DB_NAME=npm
NPM_DB_USER=npm_user
NPM_DB_PASSWORD=PAROLA_COMPLEXA_NPM
EOF

chmod 600 /opt/npm/.env
```

E.3 docker-compose.yml final

```
cat > /opt/npm/docker-compose.yml << 'EOF'
services:
  npm-app:
    image: jc21/nginx-proxy-manager:latest
    container_name: npm-app
    restart: unless-stopped
    ports:
      - '80:80'
      - '443:443'
      - '10.10.10.20:81:81'
    environment:
      TZ: ${TZ}
      DB_MYSQL_HOST: ${NPM_DB_HOST}
      DB_MYSQL_PORT: ${NPM_DB_PORT}
      DB_MYSQL_USER: ${NPM_DB_USER}
      DB_MYSQL_PASSWORD: ${NPM_DB_PASSWORD}
      DB_MYSQL_NAME: ${NPM_DB_NAME}
    volumes:
      - ./data:/data
      - ./letsencrypt:/etc/letsencrypt
      - ./logs:/data/logs
    extra_hosts:
      - 'host.docker.internal:host-gateway'
    networks:
      - npm_net

networks:
  npm_net:
    name: npm_net
```

```
driver: bridge
ipam:
  config:
    - subnet: 172.30.10.0/24
EOF
```

Maparea ./logs:/data/logs separa logurile Nginx Proxy Manager intr-un volum/director extern dedicat. Chiar daca ./data:/data include implicit subdirectorul logs, maparea explicita ./logs:/data/logs simplifica backup-ul si auditarea logurilor.

E.4 Pornire si verificare NPM

```
cd /opt/npm

docker compose config
docker compose pull
docker compose up -d

docker compose ps
docker logs -f npm-app

# Verificare porturi publice si admin local
ss -lntp | egrep ':80|:443|:81'

# Verificare loguri mapate extern
ls -la /opt/npm/logs
```

ANEXA F. Certificate SSL in Nginx Proxy Manager prin Cloudflare DNS Challenge

Certificatele SSL pentru domeniile WordPress se genereaza in Nginx Proxy Manager, nu in Nginx backend. Nginx backend ramane HTTP local pe 127.0.0.1:8081/8082. Pentru emiterea certificatelor se foloseste DNS Challenge cu token Cloudflare la nivel de zona DNS.

F.1 Creare API Token in Cloudflare

In Cloudflare se creeaza un API Token dedicat exclusiv pentru DNS Challenge. Tokenul trebuie sa aiba permisiuni minime, nu Global API Key.

Setare	Valoare recomandata
Permissions	Zone - DNS - Edit
Zone Resources	Include - Specific zone - ernu.sec / domeniul real
Client IP Filtering	Optional: IP public al serverului NPM
TTL token	Optional, conform politicii interne

F.2 Introducere certificat in NPM

In interfata Nginx Proxy Manager: SSL Certificates -> Add SSL Certificate -> Let's Encrypt. Se introduc domeniile, adresa de e-mail si se activeaza optiunea Use a DNS Challenge. Providerul selectat este Cloudflare.

```
dns_cloudflare_api_token = TOKEN_CLOUDFLARE_DNS_CHALLENGE
```

Tokenul Cloudflare nu se introduce in docker-compose.yml si nu se publica in GitHub. El se introduce doar in interfata NPM sau intr-un mecanism securizat de secrete, in functie de procedura operationala folosita.

F.3 Proxy Host cu certificat

Camp NPM	Site 1	Site 2
Domain Names	wp01.ernu.sec	wp02.ernu.sec
Scheme	http	http
Forward Hostname/IP	127.0.0.1	127.0.0.1
Forward Port	8081	8082
SSL Certificate	Let's Encrypt Cloudflare	Let's Encrypt Cloudflare
Force SSL	Enabled	Enabled
HTTP/2 Support	Enabled	Enabled
HSTS	Enabled dupa validare	Enabled dupa validare

ANEXA G. Instalare WordPress backend local si mapare prin NPM

Aceasta anexa completeaza instalarea WordPress pentru modelul cu Nginx Proxy Manager in fata. WordPress nu expune direct HTTPS. Backend-ul local asculta HTTP pe 127.0.0.1, iar TLS este terminat in NPM.

G.1 Creare utilizator sistem si directoare

```
useradd -r -s /usr/sbin/nologin -d /var/www/wp01 wp01
mkdir -p /var/www/wp01
chown -R wp01:wp01 /var/www/wp01
```

G.2 Descarcare WordPress

```
cd /tmp
wget https://wordpress.org/latest.tar.gz
tar -xzf latest.tar.gz
rsync -av wordpress/ /var/www/wp01/
chown -R wp01:wp01 /var/www/wp01
find /var/www/wp01 -type d -exec chmod 755 {} \;
find /var/www/wp01 -type f -exec chmod 644 {} \;
```

G.3 Configurare wp-config.php

```
cp /var/www/wp01/wp-config-sample.php /var/www/wp01/wp-config.php
nano /var/www/wp01/wp-config.php
```

```
# Setari principale
define('DB_NAME', 'wp01_db');
define('DB_USER', 'wp01_user');
define('DB_PASSWORD', 'PAROLA_COMPLEXA_WP01');
define('DB_HOST', 'localhost');
define('DB_CHARSET', 'utf8mb4');
define('DB_COLLATE', '');
```

```
# Recomandari hardening
define('DISALLOW_FILE_EDIT', true);
define('FORCE_SSL_ADMIN', true);
define('FS_METHOD', 'direct');
```

G.4 Nginx backend site 1 pe port local 8081

```
cat > /etc/nginx/sites-available/wp01-backend.conf << 'EOF'
server {
    listen 127.0.0.1:8081;
    server_name wp01.ernu.sec;
    root /var/www/wp01;
    index index.php index.html;

    location / {
        try_files $uri $uri/ /index.php?$args;
    }

    location ~ /\.php$ {
        include fastcgi_params;
        fastcgi_pass unix:/run/php/php8.3-wp01.sock;
        fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
    }

    location = /xmlrpc.php { deny all; }
    location ~* /wp-content/uploads/.*.php$ { deny all; }
    location ~ /\. { deny all; }
}
EOF

ln -sf /etc/nginx/sites-available/wp01-backend.conf /etc/nginx/sites-enabled/wp01-backend.conf
nginx -t
systemctl reload nginx
```

G.5 Verificari finale

```
# NPM trebuie sa asculte public pe 80 si 443
ss -lntp | egrep ':80|:443'

# Interfata admin NPM trebuie sa fie accesibila doar din LAN/VPN
ss -lntp | grep ':81'

# Backend WordPress trebuie sa fie doar local
ss -lntp | grep ':8081'

# Test local backend
curl -I http://127.0.0.1:8081
```

```
# Test prin domeniu dupa configurarea NPM si Cloudflare
curl -I https://wp01.ernu.sec
```

ANEXA H. Acces management NPM din reteaua locala

Interfata de management Nginx Proxy Manager nu trebuie publicata pe Internet, dar trebuie sa fie accesibila administratorilor din reteaua locala. Din acest motiv portul 81 nu se mapeaza pe localhost, ci pe adresa LAN a serverului, iar accesul se restrictioneaza prin firewall doar la subnetul administrativ.

Model recomandat: NPM admin pe 10.10.10.20:81, acces permis doar din LAN/VPN, blocat din Internet.

H.1 Mapare corecta in docker-compose.yml

```
ports:
  - '80:80'
  - '443:443'
  - '10.10.10.20:81:81'
```

H.2 Reguli firewall pentru acces LAN

```
# Permite acces la interfata NPM doar din reteaua locala de administrare
ufw allow from 10.10.10.0/24 to any port 81 proto tcp comment 'NPM admin LAN only'

# Optional: daca serverul are si interfata publica, blocati explicit accesul public la portul 81
ufw deny 81/tcp comment 'Block public NPM admin'

# Verificare reguli
ufw status numbered
ss -lntp | grep ':81'
```

H.3 URL de administrare

```
http://10.10.10.20:81
```

```
# Acces permis doar din LAN/VPN, nu din Internet.
```